

Phase 2 – Managed Node Groups

Objective

In Phase 1, we created the EKS control plane and established secure connectivity from an administration EC2 instance.

In Phase 2, we add worker nodes to provide compute capacity for Kubernetes workloads.

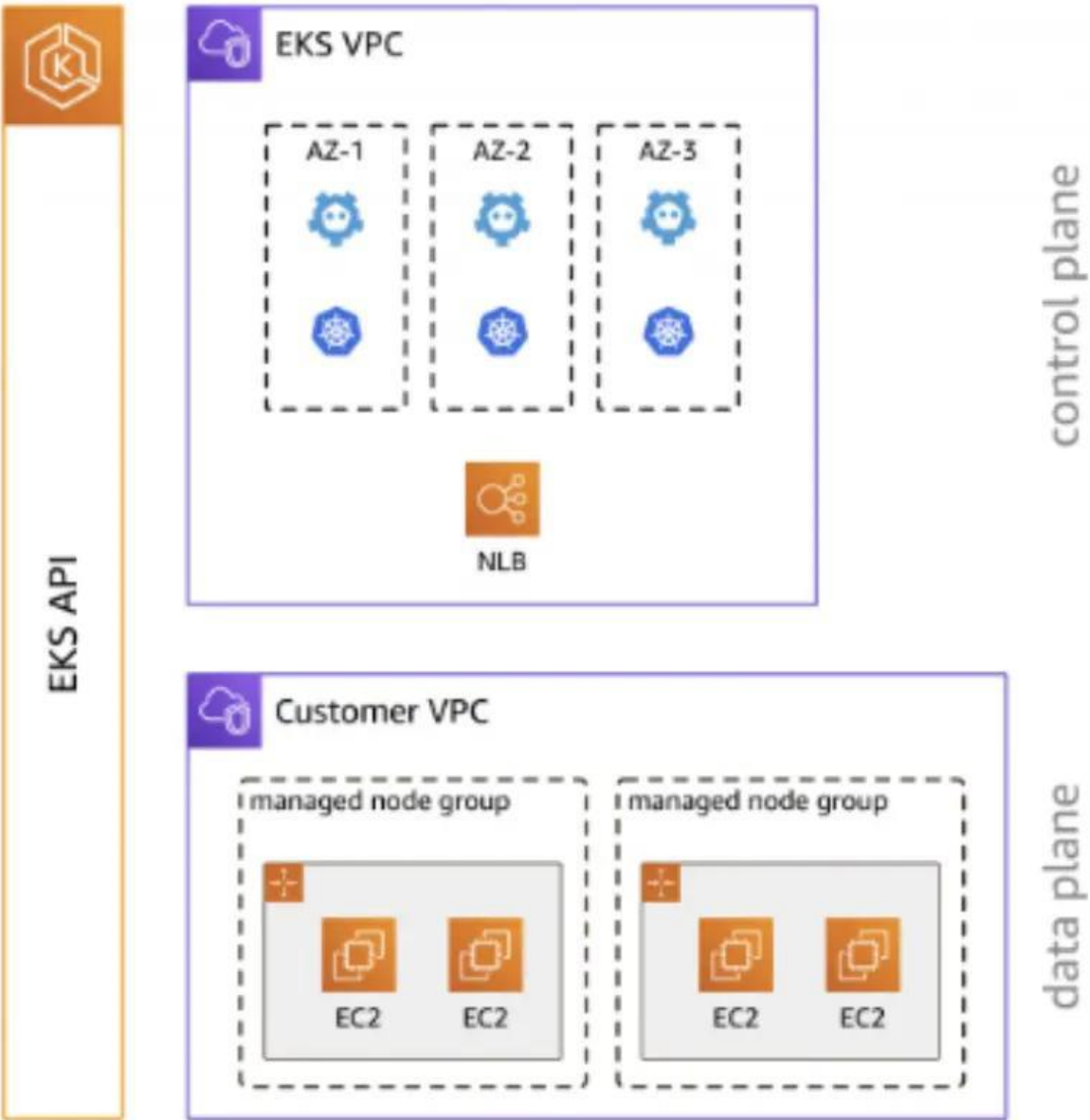
Without worker nodes:

Control Plane	= Running
CoreDNS Pods	= Pending
Applications	= Cannot Run

After creating a Managed Node Group:

Control Plane	= Running
Worker Nodes	= Running
CoreDNS Pods	= Running
Applications	= Can Run

Visual Reference — What We Are Building



Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2 (AL2_x86_64) ▼

Capacity type

Select the capacity purchase option for this node group.

On-Demand ▼

Instance types [Info](#)

Select instance types you prefer for this node group.

Select ▼

t3.micro ✕
vCPU: 2 vCPUs Memory: 1 GiB Network: Up to 5 Gigabit Max ENI: 2 Max IPs: 4

Disk size

Select the size of the attached EBS volume for each node.

20 GiB

Node group scaling configuration

Desired size

Set the desired number of nodes that the group should launch with initially.

2 nodes

Minimum size

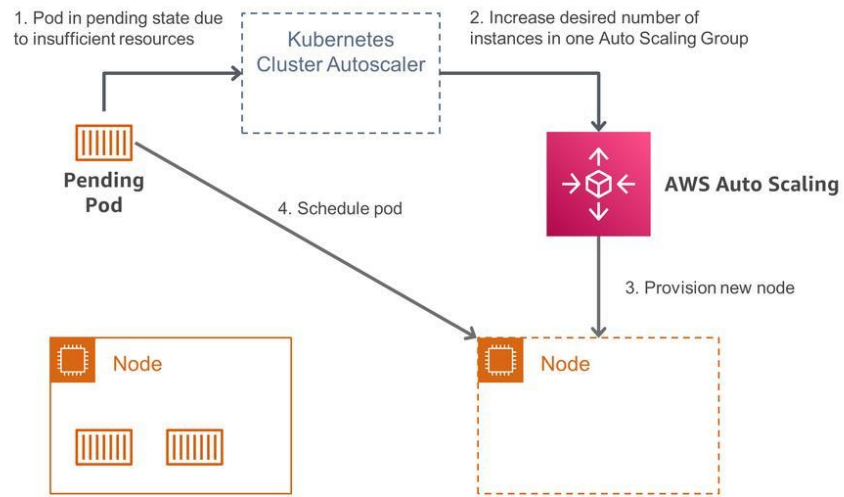
Set the minimum number of nodes that the group can scale in to.

2 nodes

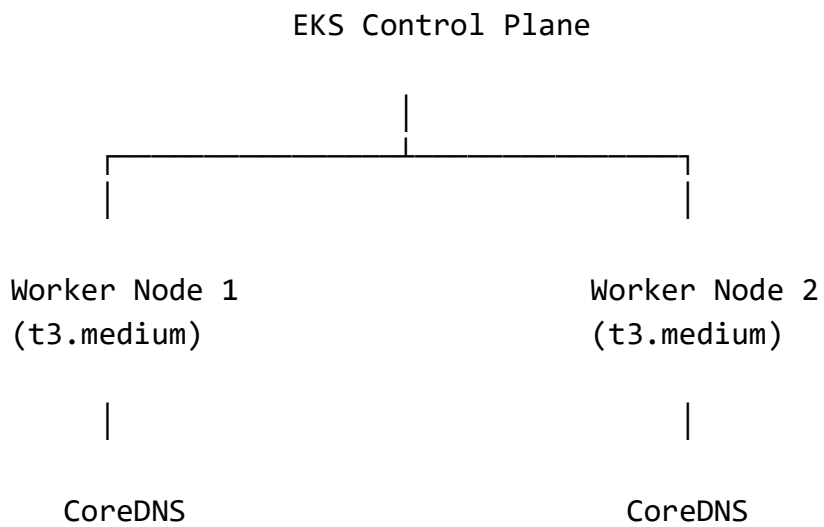
Maximum size

Set the maximum number of nodes that the group can scale out to.

2 nodes



Architecture After Phase 2



Step 1 – Configure Node Group

Reference Screen:

The screenshot shows the 'Configure Node Group' page in the Amazon EKS console. The left sidebar shows the navigation menu with 'Clusters' selected. The main content area is titled 'Configure Node Group' and includes a progress bar with four steps: 'Step 1: Configure Node Group' (active), 'Step 2: Set compute configuration', 'Step 3: Set scaling configuration', and 'Step 4: Review and create'. The page contains several configuration sections with red boxes and arrows pointing to specific fields, each with a numbered Chinese annotation:

- Group configuration:**
 - Name:** A red box highlights the 'ng1-by-console' field with the annotation '1. 定义 Cluster Name'.
 - Node IAM Role Name:** A red box highlights the dropdown menu showing 'eksctl-eksdemo-nodegroup-ng-546-NodeInstanceRole-172ZU91D5PC0P' with the annotation '2. 为Node选择 IAM Role'. Below this is a warning message: 'The selected role must not be used by a self-managed node group as this could lead to a service interruption upon Managed Node Group deletion. Learn more'.
 - Subnets:** A red box highlights the dropdown menu showing 'All subnets' with the annotation '3. 选择Node所在的子网'.
- Allow remote access to nodes:**
 - A red box highlights the 'Allow remote access to nodes' checkbox with the annotation '4. 选择是否允许SSH到Node上'.
- SSH key pair:** A red box highlights the dropdown menu showing 'pk-bjs'.
- Allow remote access from:** Two radio buttons are present: 'All' (selected) and 'Selected security groups'.
- Tags and labels:**
 - Tags:** A red box highlights the 'Add tag' button with the annotation '5. 为Node Group添加Tag'.
 - Kubernetes labels:** A red box highlights the 'Add label' button with the annotation '6. 为Node Group添加 labels'.

Set compute and scaling configuration

Node Group compute configuration

These properties cannot be changed after the Node Group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2 (AL2_x86_64)

Instance type [Info](#)

Select the EC2 instance type for nodes.

t3.large

Disk size

Select the size of the attached EBS volume for each node.

100

GiB

Node Group scaling configuration

Minimum size

Set the minimum number of nodes that the group can scale in to.

2

nodes

Maximum size

Set the maximum number of nodes that the group can scale out to.

2

nodes

Desired size

Set the desired number of nodes that the group should launch with initially.

2

nodes

[IAM](#) > [Roles](#) > [Create role](#)

Step 1
[Select trusted entity](#)

Step 2
Add permissions

Step 3
[Name, review, and create](#)

Add permissions

Permissions policies (Selected 3/771)

Choose one or more policies to attach to your new role.

Filter policies by property or policy name and press enter.

1 match

"AmazonEKSWorkerNodePolicy" X

Clear filters

☒	Policy name ↗	Type	Description
☒	 AmazonEKSWorkerNodePolicy	AWS managed	This policy allows Amazon EKS worker nodes to connect to Amazon EKS Clusters.

Node Group Name

Selected:

mani-node-group

Purpose:

Logical identifier for the worker node pool.

Node IAM Role

Created:

AmazonEKSAutoNodeRole

Purpose:

Worker nodes require permissions to:

Join the cluster

Pull images from ECR

Manage networking

Use AWS services

Why Nodes Need IAM Permissions

Worker nodes are EC2 instances.

Kubernetes nodes must:

Authenticate with EKS

Pull container images

Attach networking resources

Access AWS APIs

Without these permissions:

EC2 launches

Node never joins cluster

AWS Managed Policies Explained

Typical policies include:

AmazonEKSWorkerNodePolicy

Allows:

Node registration with EKS

Cluster communication

AmazonEC2ContainerRegistryPullOnly

Allows:

Pull images from Amazon ECR

AmazonEKS_CNI_Policy

Allows:

Pod networking

ENI management

IP assignment

AmazonSSMManagedInstanceCore

Allows:

Systems Manager access

Remote administration

Kubernetes Labels

Configured:

team=platform

environment=dev

Purpose:

Labels are Kubernetes metadata used for:

Scheduling

Grouping

Node Selection

Affinity Rules

Example:

nodeSelector:

team: platform

AWS Tags vs Kubernetes Labels

AWS Tags

Used by AWS

Kubernetes Labels

Used by Kubernetes

Billing
Cost allocation
Automation

Scheduling
Pod placement
Resource selection

Step 2 – Compute Configuration

Reference Screen:

Set compute and scaling configuration

Node Group compute configuration

These properties cannot be changed after the Node Group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2 (AL2_x86_64) ▼

Instance type [Info](#)

Select the EC2 instance type for nodes.

t3.large ▼

Disk size

Select the size of the attached EBS volume for each node.

100

GiB

Node Group scaling configuration

Minimum size

Set the minimum number of nodes that the group can scale in to.

2

nodes

Maximum size

Set the maximum number of nodes that the group can scale out to.

2

nodes

Desired size

Set the desired number of nodes that the group should launch with initially.

2

nodes

Set compute and scaling configuration

Node Group compute configuration

These properties cannot be changed after the Node Group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2 (AL2_x86_64)

Capacity type

Select the capacity purchase option for this Node Group.

Spot

Instance types [Info](#)

Select instance types you prefer for this Node Group.

Select

t2.medium

vCPU: Up to 2 vCPUs memory: 4.0 GiB

Disk size

Select the size of the attached EBS volume for each node.

20

GiB

Node Group scaling configuration

Minimum size

Set the minimum number of nodes that the group can scale in to.

2

nodes

Maximum size

Set the maximum number of nodes that the group can scale out to.

2

nodes

Desired size

Set the desired number of nodes that the group should launch with initially.

2

nodes

Cancel

Previous

Next

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2023 (x86_64) Standard (AL2023_x86_64_STANDARD)	▲
Amazon Linux 2 (AL2_x86_64)	
Amazon Linux 2 GPU Enabled (AL2_x86_64_GPU)	
Amazon Linux 2 Arm (AL2_ARM_64)	
Bottlerocket (BOTTLEROCKET_x86_64)	
Bottlerocket Arm (BOTTLEROCKET_ARM_64)	
Bottlerocket NVIDIA (BOTTLEROCKET_x86_64_NVIDIA)	
Bottlerocket Arm NVIDIA (BOTTLEROCKET_ARM_64_NVIDIA)	
Amazon Linux 2023 (x86_64) Standard (AL2023_x86_64_STANDARD)	✓
Amazon Linux 2023 (arm64) Standard (AL2023_ARM_64_STANDARD)	
Windows Server 2019 Core (WINDOWS_CORE_2019_x86_64)	
Windows Server 2022 Core (WINDOWS_CORE_2022_x86_64)	
Windows Server 2019 Full (WINDOWS_FULL_2019_x86_64)	
Windows Server 2022 Full (WINDOWS_FULL_2022_x86_64)	

AMI Type

Selected:

Amazon Linux 2023

Purpose:

Pre-configured operating system optimized for Kubernetes.

Includes:

kubelet
container runtime
AWS integrations

Capacity Type

Selected:

On-Demand

Reason:

Predictable
No interruptions
Best for learning

Instance Type

Selected:

t3.medium

Resources:

2 vCPU
4 GB RAM

Cluster Capacity:

2 Nodes
=
4 vCPU
8 GB RAM

Disk Size

Selected:

20 GB

Per node.

Total:

20 GB × 2 Nodes
=
40 GB

Scaling Configuration

Configured:

Desired = 2

Minimum = 2

Maximum = 3

Meaning:

Desired

Launch 2 nodes immediately

Minimum

Never scale below 2

Maximum

Can scale up to 3

Node Auto Repair

Enabled:

Yes

Purpose:

Automatically replaces unhealthy nodes.

Example:

Node becomes NotReady



EKS detects failure



Node terminated



Replacement node launched

Step 3 – Networking

Reference Screen:

Step 1
Configure node group

Step 2
Set compute and scaling
configuration

Step 3
Specify networking

Step 4
Review and create

Specify networking

Node group network configuration

These properties cannot be changed after the node group is created.

Subnets [Info](#)

Specify the subnets in your VPC where your nodes will run. To create a new subnet, go to the corresponding page in the [VPC console](#).

Select subnets

subnet-042c40fe0bf94c112 X

subnet-070b66054208240c6 X

subnet-06651ed99d6fe1e94 X

subnet-0bbb12f3dae1bfcef X

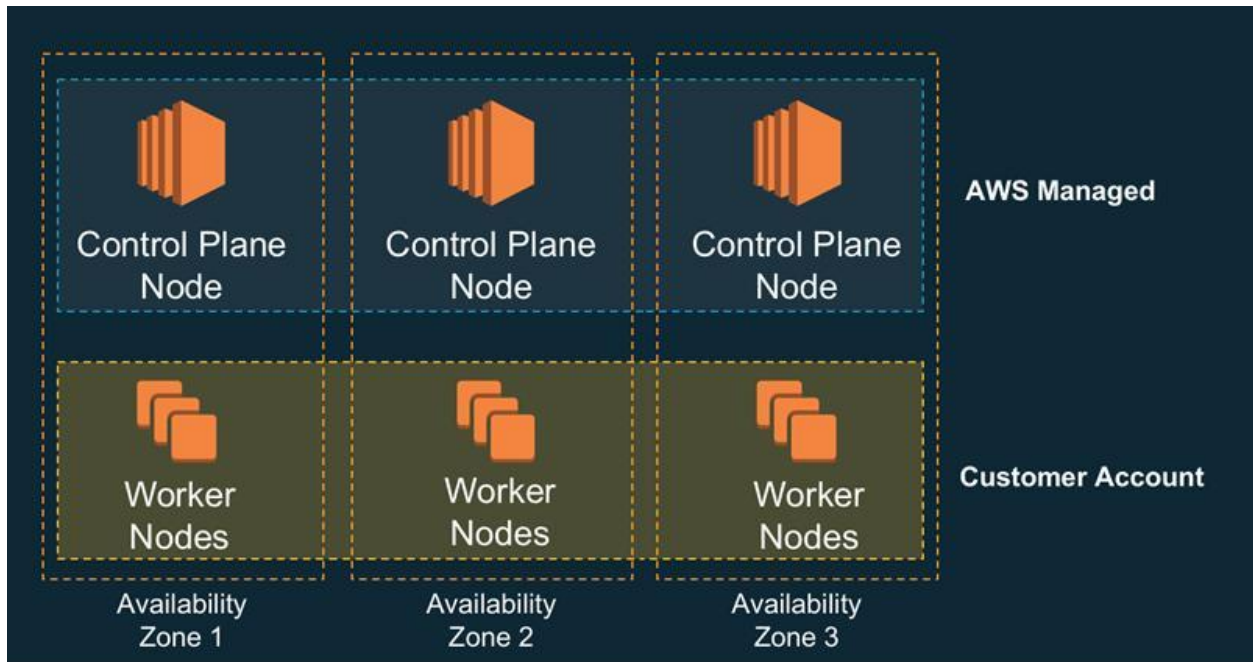
subnet-02659c151cbc73d11 X

☐ Configure SSH access to nodes [Info](#)

Cancel

Previous

Next



EKS > Clusters > mng > Add node group

Step 1
Configure node group

Step 2
Set compute configuration

Step 3
Set scaling configuration

Step 4
Review and create

Configure node group

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster. [Info](#)

Group configuration - cannot be changed after node group creation

Name
Assign a unique name for this node group

Node IAM Role Name [Info](#)
Select the IAM Role that will be used by the nodes.

Subnets [Info](#)
Specify the subnets in your VPC where your nodes will run

Add subnets

subnet-00a8eca84118b6e35 mng-eks-1-14-hausenbl-11-12-2019-NODE-VPC-Subnet01 ✕
us-west-2a 192.168.64.0/28

subnet-00d3bc37b9805e196 mng-eks-1-14-hausenbl-11-12-2019-NODE-VPC-Subnet03 ✕
us-west-2c 192.168.192.0/28

subnet-0965334f9fa2225e8 mng-eks-1-14-hausenbl-11-12-2019-NODE-VPC-Subnet02 ✕
us-west-2b 192.168.128.0/28

☐ **Allow remote access to nodes** [Info](#)
Without remote access enabled you will not be able to directly connect to nodes after they are created.

Tags and labels

Selected Subnets

ap-south-1a
ap-south-1b
ap-south-1c

Purpose:

Deploy nodes across multiple Availability Zones.

Benefits:

High Availability
Fault Tolerance
Production Best Practice

Remote Access

Configured:

Disabled

Reason:

Cluster administration performed through:

kubect1

from the dedicated administration EC2 instance.

Final Configuration Summary

Node Group Name:

mani-node-group

Node Role:

AmazonEKSAutoNodeRole

Labels:

team=platform

environment=dev

Capacity Type:

On-Demand

Instance Type: t3.medium (**change to eligible one if using free tier account**)

Disk:

20 GB

Desired:

2

Minimum:

2

Maximum:

3

Node Auto Repair:

Enabled

Subnets:

3 AZs

Remote Access:

Disabled

Expected Validation

Verify Nodes

```
kubectl get nodes
```

Expected:

2 Nodes

STATUS = Ready

Verify System Pods

```
kubectl get pods -A
```

Expected:

kube-system

coredns-xxxxx

Running

coredns-yyyyy

Running

Resources AWS Will Create:

Managed Node Group



Auto Scaling Group



Launch Template



2 EC2 Instances



Join EKS Cluster